

# Operational Detection of Guest-to-Host Compromise via QEMU Process Telemetry

Joshua Buckwald

*Department of Cybersecurity*

*New York University*

New York, United States

jsb518@nyu.edu

Presentation: [https://stream.nyu.edu/media/JoshBuckwaldPresentation\\_04.26.26/1\\_q6c41q30](https://stream.nyu.edu/media/JoshBuckwaldPresentation_04.26.26/1_q6c41q30)

**Abstract**—This research explores the operational detection of guest-to-host compromise in virtualized environments using host-level telemetry of the Quick Emulator (QEMU) process. We examine published literature surveying modern virtualization threats, as well as the limitations of existing approaches such as Virtual Machine Introspection (VMI). A controlled experiment was conducted to simulate post-exploitation behavior through command execution within the QEMU process, and host-based monitoring was used to evaluate detection capability. The results suggest that QEMU process telemetry may provide a fast and interpretable signal for detecting compromise at the hypervisor boundary, offering a practical complement to existing detection methods.

**Index Terms**—virtualization, QEMU, virtual machine introspection, hypervisor compromise

## I. INTRODUCTION

Modern cloud and enterprise infrastructure relies heavily on virtualization to provide strong isolation between workloads. In Kernel-based Virtual Machine (KVM) environments, this isolation is enforced through the hypervisor and its associated virtualization processes, most notably Quick Emulator (QEMU). Guest-to-host compromise (commonly referred to as VM escape) represents a critical class of attacks capable of violating this isolation boundary and granting an attacker control over the host system. Such compromises undermine the foundational security assumptions of multi-tenant environments and can enable lateral movement, data exfiltration, and persistent control of underlying infrastructure.

Despite the severity of this threat, operational detection of guest-to-host compromise remains a challenging problem. Traditional detection strategies must assume that telemetry originating from the guest operating system may be untrustworthy, as an attacker with sufficient control can manipulate logs, suppress signals, or otherwise obscure their activity. As a result, research has focused on approaches such as Virtual Machine Introspection (VMI), which observes guest behavior externally through low-level memory and hardware state. While VMI provides strong isolation from attacker manipulation, it introduces the problem known as the “semantic gap”, requiring reconstruction of meaningful operating system behavior from raw memory and hardware state. This process increases analytical complexity and can delay detection, limiting its effectiveness in real-time operational settings.

While the underlying assumptions regarding the trustworthiness of guest telemetry are generally valid, they do not fully apply at the boundary between guest and host. In order to achieve guest-to-host compromise, an attacker must induce observable behavior in host components such as the QEMU process. Unlike guest telemetry, which can be manipulated by an attacker, this approach focuses on the host level effects of guest activity as they manifest in the QEMU process. These effects are significantly more difficult to suppress or falsify without prior control of the host system, creating an opportunity for detection at the moment of boundary crossing, before traditional indicators of compromise become visible.

This research explores this opportunity by observing host level telemetry of the QEMU process. As the user-space component responsible for virtual machine execution, the QEMU process represents the first host-side boundary that must be crossed during a guest-to-host compromise. We hypothesize that anomalous behavior at this boundary, such as unexpected process execution or access to sensitive host resources, can provide a direct and timely signal of compromise. To evaluate this approach, we conduct a controlled experiment in which post-exploitation behavior is simulated through command execution within the QEMU process, and detection is performed using host-based monitoring.

The organization of this paper is as follows. Section II reviews related research on virtualization security and detection techniques, including VMI-based approaches. Section III describes the experimental methodology used to simulate guest-to-host compromise and collect host-level telemetry. Section IV presents the results of the experiment and compares QEMU process telemetry with VMI-style observation. Section V concludes the paper and outlines directions for further research.

## II. RELATED RESEARCH

Mishra et al. [1] provide a comprehensive survey of intrusion detection techniques in cloud environments, highlighting the limitations of traditional host based and network based intrusion detection systems in virtualized settings. The authors emphasize the need for virtualization aware detection approaches, particularly those that leverage hypervisor visibility such as VMI. However, they also identify challenges related to

performance, scalability, and deployment complexity, noting that many existing solutions struggle to provide timely and operationally practical detection. This work establishes the need for more efficient and practical detection strategies, which this research addresses through a lightweight approach based on observing QEMU process behavior on the host.

Garfinkel and Rosenblum [2] introduced Virtual Machine Introspection as a method for observing guest system behavior from outside the virtual machine using low level memory and hardware state. This approach provides strong isolation from attacker manipulation, as it does not rely on potentially compromised guest telemetry. However, VMI’s requirement to reconstruct operating system behavior from raw memory and hardware state increases analytical complexity and can delay detection, limiting its effectiveness in real time operational environments. In contrast, this research avoids semantic reconstruction by focusing on directly observable behavior in the QEMU process.

Zhang et al. [3] propose a detection method using KVM event sequence data to identify VM memory sharing lateral movement attacks through machine learning classification. Their approach captures host level signals generated during VM execution and demonstrates that these features can effectively distinguish between normal and attack behavior with minimal performance overhead. While their work focuses on lateral movement and assumes a trusted hypervisor, it highlights the value of host observable virtualization events as a detection surface. This research extends that idea by applying host level monitoring specifically to guest-to-host compromise scenarios, focusing on QEMU process behavior as an early indicator of VM escape activity rather than post-compromise lateral movement.

### III. METHODOLOGY

To evaluate the effectiveness of QEMU process telemetry for detecting guest to host compromise, a controlled experiment was conducted using a QEMU based virtual machine on a Linux host. Host level monitoring was implemented using auditd to capture process execution and access to sensitive resources, including `/etc/shadow` and `/root`. Analysis focused on the QEMU process associated with the running virtual machine, with no reliance on telemetry from within the guest.

Rather than exploiting a specific vulnerability, the experiment simulated post-exploitation behavior by injecting a command execution payload directly into the QEMU process using a debugger. The payload executed non-interactive shell commands that accessed sensitive host resources, generating observable host side effects consistent with a successful escape. Detection was performed by identifying anomalous process activity, including shell execution and privileged file access originating from the QEMU process.

Effectiveness was evaluated using detection timing and detection phase. Detection timing was measured as the interval between payload execution and the appearance of corresponding audit log entries. Detection phase was assessed using

```
type=EXECVE msg=audit(...): argc=4 a0="sh"
a1="-c" ...
type=SYSCALL msg=audit(...): arch=c00000b7
syscall=221
success=yes uid=0 tty=(none) ...
comm="sh" exe="/usr/bin/dash"
```

Fig. 1. Representative audit event (simplified) captured during the experiment.

TABLE I  
COMPARISON OF QEMU TELEMETRY AND VMI

| Method         | MTTD                   | Kill Chain Phase      |
|----------------|------------------------|-----------------------|
| QEMU telemetry | Near real-time         | Exploitation          |
| VMI-based      | Delayed (semantic gap) | Actions on Objectives |

the Lockheed Martin Cyber Kill Chain model to determine whether detection occurred before persistence or lateral movement.

### IV. RESULTS

The experiment showed that anomalous behavior originating from the QEMU process can be detected using host level telemetry at the moment of guest-to-host compromise. Audit logs captured non-interactive shell execution and access to sensitive host resources immediately following payload execution.

Specifically, `/bin/sh` was observed executing as a child process of QEMU without an associated terminal session and with elevated privileges. Additional events showed access attempts to `/etc/shadow` and `/root`, behavior that deviates from normal QEMU operation. A real audit event captured during the experiment is shown in Fig. 1.

Detection occurred immediately after payload execution, indicating near real time visibility into compromise related activity. Using the Cyber Kill Chain model, this detection corresponds to the Exploitation phase, where the attacker’s actions first manifest on the host system. Table I summarizes the comparison with VMI-based approaches.

In contrast, approaches based on Virtual Machine Introspection would more likely detect activity during the Actions on Objectives phase, after the attacker has already achieved meaningful access or impact.

Inspection of QEMU memory and resource usage, as would be done in VMI based tactics, produced low level indicators that required interpretation and did not directly reveal malicious activity. These results demonstrate that QEMU process telemetry provides immediate and actionable signals of compromise, supporting its use as an early detection mechanism.

### V. CONCLUSION AND FURTHER RESEARCH

This research demonstrates that host level telemetry of the QEMU process can provide immediate and actionable indicators of guest-to-host compromise. By observing the host effects of guest activity, detection was achieved in near real time, at the Exploitation phase of the attack lifecycle, earlier than traditional approaches that rely on post-compromise indicators or deeper analysis.

These findings suggest that QEMU process monitoring represents a practical and lightweight method for improving operational detection of virtualization based attacks. When used alongside approaches such as Virtual Machine Introspection, this method can provide early warning signals that enable more timely response while preserving the benefits of deeper analysis.

Future work should focus on extending this approach to production environments through integration with real time monitoring frameworks such as eBPF or SIEM based detection pipelines. Additional research is also needed to evaluate robustness against more advanced evasion techniques and to validate performance at scale in multi-tenant environments.

#### REFERENCES

- [1] P. Mishra, E. S. Pilli, V. Varadharajan, and U. Tupakula, "Intrusion detection techniques in cloud environment: A survey," *Journal of Network and Computer Applications*, vol. 77, pp. 18–47, 2017.
- [2] T. Garfinkel and M. Rosenblum, "A virtual machine introspection based architecture for intrusion detection," in *Proceedings of the Network and Distributed Systems Security Symposium (NDSS)*, 2003.
- [3] K. Zhang, J. Ma, and C. Li, "Using KVM events to detect VM memory-sharing lateral movement attacks in a virtualized environment," in *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, 2024, doi: 10.1109/ISPA63168.2024.00150.